Tarea S2 · Tu API, tu contenedor y tus comparaciones

Entrega: antes de S3 · repositorio personal con rama `tarea-s2` (o carpeta comprimida si no hay repo).

Qué se pide

Llevar **el modelo de tu caso** al punto al que hoy llevamos el proyecto base: un servicio con un contrato que sale de la firma del modelo, errores que dicen quién se equivocó, una imagen mínima, un workflow que la prueba y una comparación de predicciones por todos los caminos.

Hay dos rutas. Elige una y dilo en la evidencia.

- **Ruta A · Online Retail II.** Partes de `04-Proyecto-Base/` y haces entrar en el servicio la **variable de la tarea de S1** (si en S1 decidiste no conservarla, úsala igualmente aquí: esta tarea trata del servicio, no de si la variable mejora el modelo).
- **Ruta B · tu caso** (Bank Marketing, como en el enunciado transversal). Partes del modelo y los datos de tu caso. El criterio es el mismo; el esquema, los límites y los casos de prueba son tuyos.

Punto de partida de la Ruta A. Crea la rama `tarea-s2` desde `develop` (que ya trae la API y el Dockerfile) e incorpora los cambios de tu rama `tarea-s1` (con `git merge tarea-s1`, o repitiendo la receta de S1). Si hay conflictos, resuélvelos y anótalo en la evidencia. Con tu variable dentro y la API sin tocar, las pruebas de la API **fallarán**: es lo esperado, y es de lo que trata el paso 2.

Pasos

1. **Reentrena** y deja el modelo en formato MLflow, con su firma y `metadatos.json` (Ruta A: `python -m ia_mlops.pipeline`).
2. **Mira cómo la API detecta el cambio.** Arranca la API con el esquema viejo y consulta `/listo` y `/predecir`. Guarda el mensaje exacto.
3. **Actualiza el contrato**: campo nuevo en `ia_mlops/api/esquemas.py` con **límites justificados por cómo se calcula la variable** (no por lo que viste al entrenar), regla de coherencia con las demás si procede, `EJEMPLO` y los archivos de `tests/datos/`.
4. **Pruebas.** Todo el conjunto en verde y, además, **al menos dos pruebas tuyas**: una que muestre un 422 para tu variable (un valor imposible) y otra que muestre una coherencia o una advertencia.
5. **Contenedor.** Construye la imagen (en un Codespace o con el CI), ejecútala y comprueba `/listo`, una predicción y que el proceso no corre como root.
6. **Workflow.** Una ejecución en verde en tu repositorio (enlace o texto del registro).
7. **Comparación.** `python -m ia_mlops.modeling.comparar --url …` contra tu **contenedor**. Si alguna diferencia no es cero, encuéntrala y explícala.
8. **Escribe la evidencia** en `evidencias/plantilla-api-y-contenedor.md`: qué rechaza tu API y qué solo avisa, y por qué.

Dónde se toca (Ruta A, receta validada con `monto_180d`)

Añadir la variable al servicio **después** de haberla añadido a FS y ML (tarea de S1) pasa por esto, en este orden:

Qué haces	Qué ves
Reentrenas con la variable nueva (`python -m ia_mlops.pipeline`) y arrancas la API **sin tocarla**	`/listo` y `/predecir` responden **503**: «El esquema de la API no coincide con la firma del modelo. Faltan en la API: ['monto_180d']…». La API no se cae ni da 500: lo dice.
Añades el campo a la clase `Cliente` (con `Field(ge=0, le=1e9, …)`) pero no al ejemplo	Fallan varias pruebas de la API (en el ensayo, **18** de las rápidas): el ejemplo y los casos ya no cumplen el contrato (`Field required`).
Añades el campo a `EJEMPLO`, a `tests/datos/cliente_ejemplo.json`, a `tests/datos/lote_ejemplo.json` y a **los diez** `tests/datos/casos/*.json`	Las pruebas vuelven a verde.
Añades la regla de coherencia (`monto_90d <= monto_180d <= monto_total`)	Un caso con 90 días mayor que 180 se rechaza con 422. Escribe tu prueba.

Archivos: `ia_mlops/api/esquemas.py` (campo, regla y `EJEMPLO`) y los JSON de `tests/datos/`. El modelo, `metadatos.json` y la firma se regeneran solos al reentrenar. **No** cambia el Dockerfile ni el workflow.

Con la Ruta A, el límite de la variable sale de `sql/fs/02_poblar_fs.sql`: una suma de importes de la ventana no puede ser negativa ni mayor que el gasto total. Un límite como «hasta el máximo que vi en train» **rechazaría clientes reales**.

Evidencia mínima

- Salida de `python -m pytest -q` (sin fallos) y de la parte nueva que escribiste.
- El mensaje 503 del paso 2 (texto exacto).
- La salida de `comparar --url` contra el contenedor.
- `docker image ls` de tu imagen y el resultado de `docker exec … id -u`.
- Enlace o registro de una ejecución del workflow en verde.
- `evidencias/plantilla-api-y-contenedor.md` completada.

Rúbrica

Ver `rubrica.md`: 20 % reproducibilidad, 40 % técnica, 40 % análisis y decisión.

Honestidad

No se evalúa solo por capturas. Declara lo que no pudiste ejecutar (por ejemplo, si no tuviste Docker) y el error exacto. Cita la ayuda que recibiste (personas o herramientas).